rails.js provides the JavaScript half, and the regular Ruby view helpers add appropriate tags to your DOM. The CoffeeScript in rails.js then listens for these attributes, and attaches appropriate handlers.

3.1 form_for

form_for is a helper that assists with writing forms. `form_for` takes a `:remote` option. It works like this:

```
<%= form_for(@article, remote: true) do |f| %>
  ...
<% end %>
```

This will generate the following HTML:

```
<form accept-charset="UTF-8" action="/articles" class="new_article"
data-remote="true" id="new_article" method="post">
  ...
</form>
```

Note the `data-remote="true"`. Now, the form will be submitted by Ajax rather than by the browser's normal submit mechanism.

You probably don't want to just sit there with a filled out `<form>`, though. You probably want to do something upon a successful submission. To do that, bind to the `ajax:success` event. On failure, use `ajax:error`. Check it out:

```
$(document).ready ->
  $("#new_article").on("ajax:success", (e, data, status, xhr) ->
    $("#new_article").append xhr.responseText
  ).on "ajax:error", (e, xhr, status, error) ->
    $("#new_article").append "<p>ERROR</p>"
```

Obviously, you'll want to be a bit more sophisticated than that, but it's a start. You can see more about the events in the jquery-ujs wiki.

Another possibility is returning javascript directly from the server side on remote calls:

```
# articles_controller
def create
  respond_to do |format|
    if @article.save
      format.html { ... }
      format.js do
        render js: <<-endjs
          alert('Article saved successfully!');
          window.location = '#{article_path(@article)}';
        endjs
      end
    else
      format.html { ... }
      format.js do
        render js: "alert('There are empty fields in the form!');"
      end
    end
  end
end
```

If javascript is disabled in the user browser, `format.html { ... }` block should be executed as fallback.

3.2 form_tag

form_tag is very similar to `form_for`. It has a `:remote` option that you can use like this:

```
<%= form_tag('/articles', remote: true) do %>
  ...
<% end %>
```

This will generate the following HTML: